

Ccache : A Rapid and Efficient Compiler Cache for GCC/G++



Ccache makes compilation very fast because it caches previous compilations and detects when these are being recompiled. It is free software that supports C, C++, Objective-C and Objective-C++. It manages the cache size automatically, is an easy install and has low overheads.

The process of building large projects that have C/C++ sources is time-consuming. Packages such as the Linux kernel, OpenSSH and FFmpeg take a lot of time to build, making it an expensive process. We do have the 'make' utility to optimise the build process, but there are times when it is ineffective (like when we 'make clean' and build from scratch). So 'make' does not take care of such cases. Also, when we extract sources from a tarball, a rebuild happens every time we do so.

A rebuild of sources is inefficient if we have already built the same sources earlier. This duplication of work causes a waste of CPU resources and development time.

The solution

The solution is ccache, a persistent (on-disk) C/C++ compiler cache that achieves fast compilation. It is an open source tool to optimise the build process of source code. It supports C/C++ and the Objective-C/C++ languages.

The features of ccache are:

- Provides statistics on hits/misses
- Transparent usage
- No side-effects on compilation output
- Flexibility with cache size management
- Simple installation and usage
- Low overhead

How it works

Ccache keeps track of the compilation of the source code. It caches the compilation of a single C/C++/Obj-C/Obj-C++ file and reuses the compiled output instead of recompiling the same source file. It uses many pieces of information to keep

track of the uniqueness of the compiled code such as:

- Source file name
- Compiler name
- Compiler size and modification time
- The current directory
- Command line options

Ccache maintains a persistent HashMap of the metadata and corresponding compiled code. Before compiling a source file, it checks its HashMap for the same file. It uses precompiled code whenever there is a match in the hash table. If it is a miss, ccache adds it to its hash.

Installation

Use the following code to install ccache:

```
$ sudo apt-get install ccache
```

Using ccache

There are two ways to use ccache:

1. You can either prefix your compilation commands with ccache, or...
2. Mask the compiler by creating a symbolic link (named as the compiler) to ccache.

Let's use the second method on Ubuntu. We need to prefix `/usr/lib/ccache` to the PATH environment variable. Please run the following on bash shell.

```
export PATH=/usr/lib/ccache:$PATH
```

The path `/usr/lib/ccache` contains symlinks for all the compilers currently installed as Debian packages.